

Tax Tracker — Architecture Summary

Ownership-level overview · Architecture, flow & invariants

1. App Entry & Execution Flow

Start sequence

- **AndroidManifest:** Single `MainActivity` (LAUNCHER), `android:supportsRtl="true"`, theme `Theme.TaxInvoiceTracker`.
- **MainActivity.onCreate():**
 - `attachBaseContext()` → `updateBaseContextLocale()` via `LanguagePreferenceManager` (locale + layout direction).
 - `TaxTrackerDatabase.getDatabase(this)` (singleton).
 - `RoomCategoryRepository(categoryDao)`, `RoomInvoiceRepository(invoiceDao)`.
 - `lifecycleScope.launch { seedInitialDataIfNeeded(...) }` (runs before/same time as UI).
 - `setContent { ... }`.

Theme & layout direction

- Theme: `TaxInvoiceTrackerTheme` (Material3, dynamic color on API 31+).
- RTL: `attachBaseContext` → `config.setLocale(savedLocale)` + `config.setLayoutDirection(savedLocale)`.
- Compose: `LocalConfiguration.current.layoutDirection` → `LayoutDirection.Rtl` when `View.LAYOUT_DIRECTION_RTL`, else `Ltr`.
- `CompositionLocalProvider(LocalLayoutDirection provides layoutDirection)` wraps entire UI.

NavController & root composable

- `rememberNavController()` inside `setContent`.
- Repositories: `remember { categoryRepository }`, `remember { invoiceRepository }` (avoids recreation on recomposition).
- Root: `TaxInvoiceTrackerTheme` → `Surface` → `TaxTrackerNavHost(navController, categoryRepository, invoiceRepository)`.

- `TaxTrackerNavHost` is the root composable.

2. Navigation Architecture

Routes & arguments

Route	Arguments	Screen
<code>category_list</code>	—	<code>CategoryListRoute</code>
<code>add_category</code>	—	<code>AddCategoryScreen</code>
<code>edit_category/{categoryId}</code>	<code>categoryId (Long)</code>	<code>EditCategoryScreen</code>
<code>invoice_list/{categoryId}</code>	<code>categoryId (Long)</code>	<code>InvoiceListScreen</code>
<code>add_invoice/{categoryId}</code>	<code>categoryId (Long)</code>	<code>AddInvoiceScreen</code>
<code>invoice_details/{invoiceId}</code>	<code>invoiceId (Long)</code>	<code>InvoiceDetailsScreen</code>
<code>edit_invoice/{invoiceId}</code>	<code>invoiceId (Long)</code>	<code>EditInvoiceScreen</code>
<code>language_settings</code>	—	<code>LanguageSettingsScreen</code>

Start destination: `Screen.CategoryList.route ( "category_list" )`.

Category ↔ Invoice flow

- Category list → tap category → `navigate(invoice_list/{categoryId})` .
- Invoice list → Add → `navigate(add_invoice/{categoryId})` .
- Invoice list → tap invoice → `navigate(invoice_details/{invoiceId})` .
- Invoice details → Edit → `navigate(edit_invoice/{invoiceId})` .
- Invoice list → Edit category → `navigate(edit_category/{categoryId})` .
- Category list → Settings → `navigate(language_settings)` .

ViewModel creation & injection

- **CategoryListViewModel:** `CategoryListViewModelFactory(categoryRepository, invoiceRepository, context)` . Shared via `getBackStackEntry(Screen.CategoryList.route)` for `AddCategory`, `EditCategory`.

- **InvoiceListViewModel:** `InvoiceListViewModelFactory(invoiceRepository, categoryRepository, context)` . Own instance for InvoiceList; shared via `getBackStackEntry(Screen.InvoiceList.route)` for AddInvoice, InvoiceDetails, EditInvoice.
- Repositories created in MainActivity, passed into TaxTrackerNavHost, then into factories.

3. Feature Architecture (Category & Invoice)

Category feature

Screen	ViewModel	Responsibilities
CategoryListScreen	CategoryListViewModel	Load categories, delete, show snackbar
AddCategoryScreen	same (shared)	Add category, unique name check
EditCategoryScreen	same (shared)	Update category, remove custom fields, check invoice usage

Repository methods: `getCategories` , `addCategory` , `updateCategory` , `deleteCategory` , `getInvoicesForCategory` (for `hasInvoicesWithFieldData`).

Invoice feature

Screen	ViewModel	Responsibilities
InvoiceListScreen	InvoiceListViewModel	Load invoices, delete, sort
AddInvoiceScreen	same (shared)	Add invoice
InvoiceDetailsScreen	same (shared)	Read-only display
EditInvoiceScreen	same (shared)	Update invoice

Repository methods: `getInvoicesForCategory` , `getInvoiceById` , `addInvoice` , `updateInvoice` , `deleteInvoice` ; `getCategories` for header & custom field titles.

4. Data Layer & Persistence

Domain vs Room

- **Domain:** Category, Invoice, PaymentStatus, DocumentType, CustomFieldType, CustomFieldDefinition, InvoiceCustomFieldValue, InvoiceImage (Models.kt).
- **Entities:** CategoryEntity (id, name, colorHex, description, customFieldTitlesJson), InvoiceEntity (id, categoryId, invoiceNumber, amount, paymentStatus, vendorName, date fields, customFieldValuesJson, documentTypeString).
- **Mapping:** Entity.toDomain() / Entity.fromDomain() in entity classes.

Repositories

- **RoomCategoryRepository:** getCategories, addCategory, updateCategory, deleteCategory.
- **RoomInvoiceRepository:** getInvoicesForCategory, getInvoiceById, addInvoice, updateInvoice, deleteInvoice.

DAOs

- **CategoryDao:** getAll, getById, insert, insertAll, update, deleteById, getCount.
- **InvoiceDao:** getByCategoryId, getById, insert, insertAll, update, deleteById, deleteByCategoryId. Invoice FK to Category with onDelete = CASCADE.

Database

- TaxTrackerDatabase v3, exportSchema = false. Migrations: 1 → 2 (documentTypeString), 2 → 3 (vendorName, issueDateEpochDay, servicePeriodStart/EndEpochDay).

Seeding

- MainActivity.seedInitialDataIfNeeded(). Runs only if categoryRepository.getCategories().isEmpty(). Inserts 8 default categories (Arnona, Electricity, Water, Gas, Phone/Internet, National Insurance, Business Expenses, Other) with localized strings and custom field titles.

5. Critical Invariants & Design Decisions

Custom field alignment

`customFieldTitles[i]` (category) ↔ `customFieldValues[i]` (invoice). Positional indexing; no schema table or field IDs.

- AddInvoice: `customFieldValues = List(categoryCustomFieldTitles.size) { "" } .`
- EditInvoice: `categoryCustomFieldTitles.mapIndexed { index, _ -> initialCustomFieldValues.getOrNull(index) ?: "" } .`

- InvoiceDetails: `categoryCustomFieldTitles.forEachIndexed { index, fieldTitle -> invoice.customFieldValues.getOrNull(index)?.takeIf { it.isNotBlank() } } .`

Max custom fields

- `Category.MAX_CUSTOM_FIELDS = 10` (Models.kt). CategoryForm shows "Add custom field" only when `customFieldTitles.size < MAX_CUSTOM_FIELDS` .

Converters

- StringListConverter: Gson JSON for List<String>.
- LocalDateConverter: LocalDate ↔ Long (epoch day).
- PaymentStatusConverter, DocumentTypeConverter: enum name String.

RTL / language

- LanguagePreferenceManager: SharedPreferences (language_prefs), keys en/he, normalizes iw → he.
- attachBaseContext applies locale before super.attachBaseContext.

Repository boundaries

- ViewModels depend on CategoryRepository and InvoiceRepository interfaces. Repositories return domain models; entities stay in data layer.

6. Architectural Risks

- **Tight coupling:** CategoryListViewModel and InvoiceListViewModel take Context for strings. InvoiceListViewModel.getDefaultDocumentType() hardcodes category name substrings.
- **Recomposition:** rememberSaveable for form state; customFieldValues in AddInvoice initialized from categoryCustomFieldTitles.size — if categories reload, list size could mismatch.
- **Migration:** exportSchema = false; no schema export. Custom field indices in JSON; migrations must preserve order. Soft-delete for fields not yet implemented.
- **Index alignment:** Removing a custom field in EditCategory shifts indices; existing invoices can have misaligned values (LAUNCH_PLAN suggests soft-delete).
- **Navigation arguments:** getBackStackEntry(Screen.InvoiceList.route) with parameterized routes; null categoryId/invoiceId triggers return@composable. InvoiceDetails/EditInvoice assume invoice exists in uiState.invoices.

7. Where Upcoming Features Should Be Implemented

Default/optional category fields (S3)

- Models.kt, CategoryForm, AddCategoryScreen, EditCategoryScreen, CategoryListViewModel, MainActivity.seedInitialDataIfNeeded, CategoryEntity for soft-delete.

CSV export (S5)

- New feature/export or core/export module; RoomCategoryRepository, RoomInvoiceRepository; Models for schema; Storage Access Framework.

Backup ZIP (S6)

- New feature/backup; build on CSV export; optional backup_metadata.json; Storage Access Framework.

Responsive layout (S4)

- AddInvoiceScreen, EditInvoiceScreen, AddCategoryScreen, EditCategoryScreen, CategoryForm, InvoiceListScreen, InvoiceDetailsScreen, CategoryListScreen; replace hardcoded dp with adaptive modifiers; ui/theme for typography.